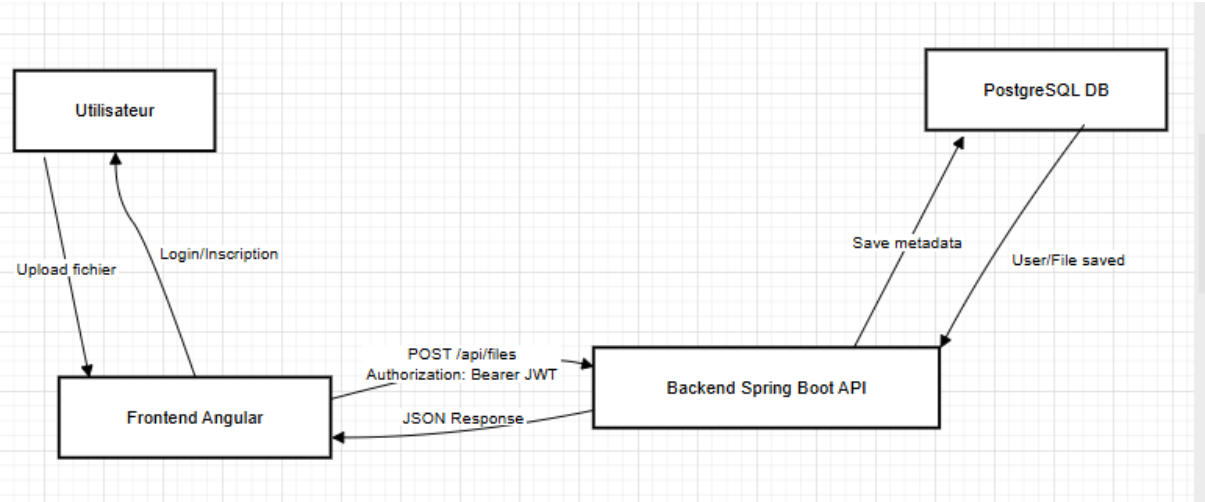
Modèle de documentation technique

Architecture de l'application 3
Choix technologiques justifiés 3
Modèle de données 5
Fonctionnement global de l'application 5
Documentation d'API 6
Sécurité et gestion des accès 6
Qualité, tests et maintenance 7
Processus d'installation et d'exécution 8
Utilisation de l'IA dans le développement 9

Architecture de l'application

Outil utilisé draw.io

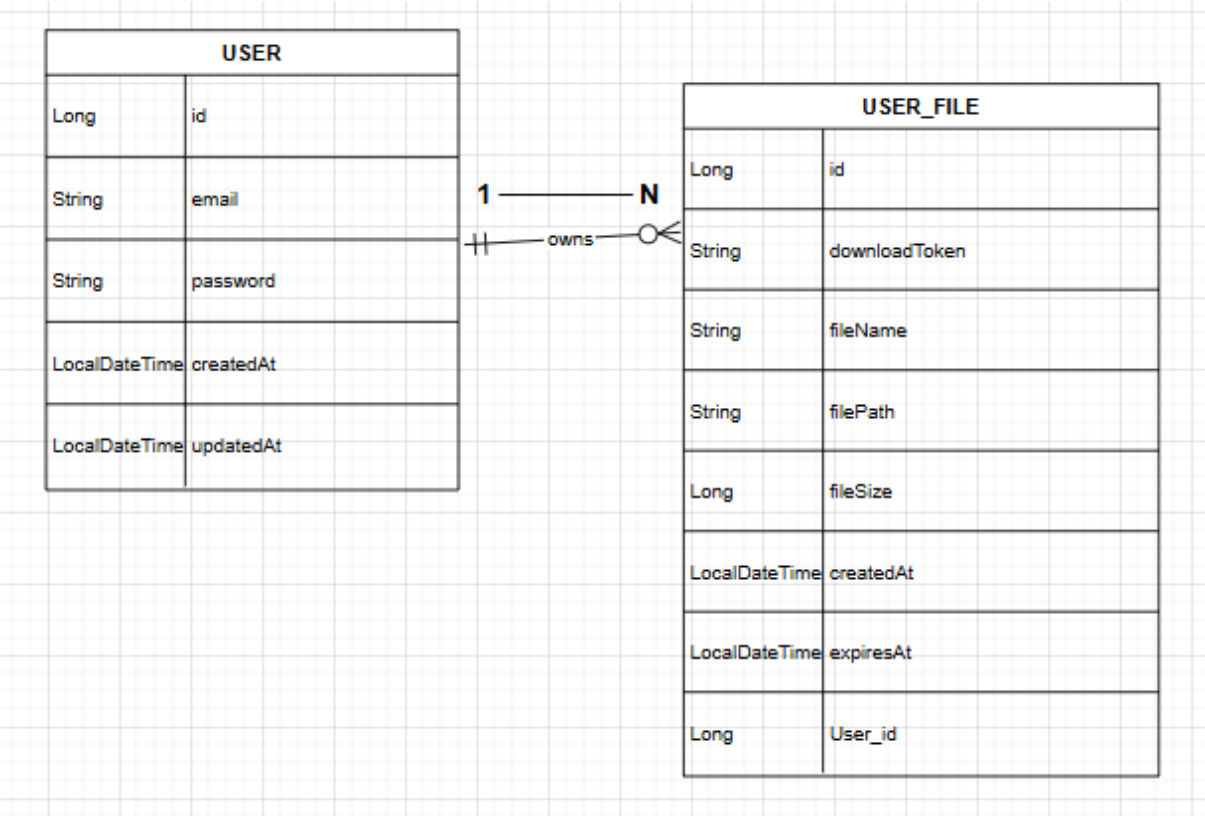


Choix technologiques justifiés

Élément	Technologie choisie	Alternatives	Justification
Langage backend	Java	Node.js, C#, PHP	Java offre robustesse, performance et forte sécurité. Idéal pour une API scalable avec Spring Boot.
Framework backend	Spring Boot	Express.js, Django, .NET	Spring Boot permet une architecture structurée (MVC), une configuration rapide.
Langage frontend	TypeScript	JavaScript	TypeScript améliore la maintenabilité grâce au typage statique et réduit les erreurs dans les projets Angular.
Framework frontend	Angular	React, Vue.js	Angular est un framework complet, structuré et avec une architecture claire
Base de données	PostgreSQL	MySQL, MongoDB	PostgreSQL garantit la cohérence des données et supporte les relations complexes
Authentification	JWT	OAuth2	JWT permet une authentification stateless,

			scalable et adaptée aux APIs REST modernes
Versioning	Git	SVN	Git permet un suivi efficace des versions, travail collaboratif et gestion des branches.
IDE	Eclipse/VS Code	IntelliJ IDEA/WebStorm	Outils complete avec autocomplétion, debug avancé et support natif Java/TypeScript
Outil de test de performance backend	K6	Gatling	k6 permet de simuler facilement des utilisateurs concurrents et d'analyser les performances des APIs REST
Outil de test de performance frontend	Lighthouse	WebPageTest	Lighthouse permet d'analyser les performances, accessibilité et SEO d'une application web
Outil pour le création de diagramme	Draw.io	Lucidchart	Draw.io permet la création de diagrammes et d'architectures d'application de façon simple et rapide.

Modèle de données



Fonctionnement global de l'application

Parcours utilisateur

1. Inscription puis connexion.
2. Réception d'un token JWT après authentification.
3. Upload d'un fichier.
4. Génération d'un lien unique de téléchargement.
5. Consultation de l'historique dans la page Mon espace.
6. Suppression manuelle d'un fichier par l'utilisateur.
7. Téléchargement du fichier soit depuis Mon espace, soit via le lien généré lors de l'upload.
8. Expiration automatique des fichiers après la durée définie

Expiration automatique

Un job backend vérifie régulièrement les fichiers expirés et les supprime :

- A **2h du matin**.
- **Au lancement du backend** pour couvrir le cas où l'application n'était pas active la nuit.

Documentation d'API

La documentation de l'API est disponible via Swagger UI :
http://localhost:8080/swagger-ui/index.html#/

Authentification

Permet de créer un compte utilisateur.

POST /api/register

Permet de s'authentifier et de récupérer un token JWT.

POST /api/login

Gestion des fichiers

Permet d'uploader un fichier.

POST /api/files/upload

Permet de télécharger un fichier via un token public.

GET /api/files/download/{token fichier}

Retourne les fichiers de l'utilisateur connecté.

GET /api/files/my-files

Supprime un fichier.

DELETE /api/files/{id}

Retourne les informations d'un fichier

GET /api/files/info/{token}

Sécurité et gestion des accès

L'application utilise un système d'authentification basé sur des **tokens (JWT)**.

- L'utilisateur s'authentifie avec ses identifiants
- Un token est généré après connexion réussie
- Ce token est envoyé avec chaque requête pour vérifier l'identité de l'utilisateur
- Le serveur valide le token avant d'autoriser l'accès aux ressources protégées

Mesures de sécurisation :

- Les mots de passe sont stockés sous forme hachée afin d'éviter leur exposition
- Les communications sont sécurisées via HTTPS pour chiffrer les échanges

- Les données envoyées par les utilisateurs sont validées et filtrées pour éviter les injections et les erreurs
- L’accès aux ressources sensibles est contrôlé côté serveur

Qualité, tests et maintenance

TESTING.md

Une stratégie de tests multi-niveaux a été implémentée :

Tests unitaires : vérification des composants critiques (JWT, services backend, composants Angular)
Tests d’intégration : validation des endpoints REST avec MockMvc et Mockito
Tests fonctionnels : contrôle des règles métier (sécurité, accès aux fichiers, validation des données)
Tests end-to-end : simulation du parcours utilisateur complet (authentification, upload, téléchargement)

Résultats :

Frontend : ~75% de couverture
Backend : ~85% global (78% code métier)

Les objectifs de couverture (70%) sont atteints.

SECURITY.md
Plusieurs mécanismes de sécurité ont été mis en place :

Authentification via JWT avec Spring Security
Autorisation basée sur l’utilisateur (accès restreint aux ressources)
Mots de passe sécurisés avec BCrypt
Protection des endpoints sensibles
Gestion des fichiers via token unique et expiration
Configuration CORS sécurisée
Protection Frontend via AuthGuard

Les choix garantissent un bon niveau de sécurité adapté à une application web moderne.

PERF.md
Des tests de performance ont été réalisés avec :
k6 pour le backend
Lighthouse pour le Frontend

Résultats principaux :
Login : ~4,8s

Upload : ~1,6s
Download : 300ms avec un fichier texte léger 1kb
Frontend : score performance 76%

MAINTENANCE.md

Des procédures de maintenance sont définies :
Mise à jour régulière des dépendances (Spring Boot, Angular)
Surveillance des vulnérabilités
Tests exécutés après chaque modification
Gestion des risques :

- Régression
- Incompatibilités
- Failles de sécurité

Processus d'installation et d'exécution

Prérequis
Les outils suivants sont nécessaires :

- Node.js (v20.x)
- npm (v10.x)
- Angular CLI (v19.x)
- Java (JDK 17)
- Apache Maven
- PostgreSQL

Commandes principales
1. Clonage du projet :

- git clone https://github.com/Abderrazzak20/Projet3_datashare.git
- cd datashare

2. Installation et lancement du backend

- mvn spring-boot:run

Le backend est accessible sur :
http://localhost:8080

3. Installation et lancement du frontend
cd frontend
npm install
npm start
Le frontend est accessible sur :
http://localhost:4200
Variables d'environnement pour le backend
Un fichier .env doit être configuré avec les variables suivantes :
JWT_SECRET_KEY=your_secret_key
JWT_EXPIRATION=3600000
DB_URL=jdbc:postgresql://localhost:5432/datashare
DB_USER=identifiant_base

DB_PASSWORD=mot_de_passe

Utilisation de l'IA dans le développement

L'intelligence artificielle a été utilisée comme outil d'assistance pour :

- Clarification de concepts techniques
- Création du script de déploiement
- Aide à la structuration de l'architecture
- Amélioration de la documentation.

Toutes les décisions finales, le code et la validation restent sous la responsabilité du développeur.

13. Conclusion

DataShare est une application complète illustrant :

- Une architecture claire et professionnelle
- Une gestion sécurisée des fichiers
- Une approche orientée qualité et maintenance.